

Using uv in Docker

Getting started

!!! tip

Check out the [`uv-docker-example``](<https://github.com/astral-sh/uv-docker-example>) project for

an example of best practices when using uv to build an application in Docker.

uv provides both *distroless* Docker images, which are useful for copying uv binaries into your own image builds, and images derived from popular base images, which are useful for using uv in a container. The distroless images do not contain anything but the uv binaries. In contrast, the derived images include an operating system with uv pre-installed.

As an example, to run uv in a container using a Debian-based image:

```
$ docker run --rm -it ghcr.io/astral-sh/uv:debian uv --help
```

Available images

The following distroless images are available:

- `ghcr.io/astral-sh/uv:latest`
- `ghcr.io/astral-sh/uv:{major}.{minor}.{patch}`, e.g., `ghcr.io/astral-sh/uv:0.12.7`
- `ghcr.io/astral-sh/uv:{major}.{minor}`, e.g., `ghcr.io/astral-sh/uv:0.12` (the latest patch version)

And the following derived images are available:

- Based on `alpine:3.23`:
 - `ghcr.io/astral-sh/uv:alpine`
 - `ghcr.io/astral-sh/uv:alpine3.23`
- Based on `alpine:3.22`:
 - `ghcr.io/astral-sh/uv:alpine3.22`
- Based on `debian:trixie-slim`:
 - `ghcr.io/astral-sh/uv:debian-slim`
 - `ghcr.io/astral-sh/uv:trixie-slim`
- Based on `buildpack-deps:trixie`:
 - `ghcr.io/astral-sh/uv:debian`
 - `ghcr.io/astral-sh/uv:trixie`
- Based on `dhi.io/alpine-base:3.23`:
 - `ghcr.io/astral-sh/uv:alpine-dhi`
 - `ghcr.io/astral-sh/uv:alpine3.23-dhi`
- Based on `dhi.io/debian-base:trixie-debian13`:
 - `ghcr.io/astral-sh/uv:debian-dhi`
 - `ghcr.io/astral-sh/uv:trixie-dhi`
- Based on `dhi/python:3.x`:
 - `ghcr.io/astral-sh/uv:python3.14-dhi`
 - `ghcr.io/astral-sh/uv:python3.13-dhi`
 - `ghcr.io/astral-sh/uv:python3.12-dhi`
 - `ghcr.io/astral-sh/uv:python3.11-dhi`
 - `ghcr.io/astral-sh/uv:python3.10-dhi`
- Based on `python3.x-alpine`:
 - `ghcr.io/astral-sh/uv:python3.15-rc-alpine`

- ghcr.io/astral-sh/uv:python3.15-rc-alpine3.23
- ghcr.io/astral-sh/uv:python3.14-alpine
- ghcr.io/astral-sh/uv:python3.14-alpine3.23
- ghcr.io/astral-sh/uv:python3.13-alpine
- ghcr.io/astral-sh/uv:python3.13-alpine3.23
- ghcr.io/astral-sh/uv:python3.12-alpine
- ghcr.io/astral-sh/uv:python3.12-alpine3.23
- ghcr.io/astral-sh/uv:python3.11-alpine
- ghcr.io/astral-sh/uv:python3.11-alpine3.23
- ghcr.io/astral-sh/uv:python3.10-alpine
- ghcr.io/astral-sh/uv:python3.10-alpine3.23
- ghcr.io/astral-sh/uv:python3.9-alpine
- ghcr.io/astral-sh/uv:python3.9-alpine3.22
- Based on python3.x-trixie:
 - ghcr.io/astral-sh/uv:python3.15-rc-trixie
 - ghcr.io/astral-sh/uv:python3.14-trixie
 - ghcr.io/astral-sh/uv:python3.13-trixie
 - ghcr.io/astral-sh/uv:python3.12-trixie
 - ghcr.io/astral-sh/uv:python3.11-trixie
 - ghcr.io/astral-sh/uv:python3.10-trixie
 - ghcr.io/astral-sh/uv:python3.9-trixie
- Based on python3.x-slim-trixie:
 - ghcr.io/astral-sh/uv:python3.15-rc-trixie-slim
 - ghcr.io/astral-sh/uv:python3.14-trixie-slim
 - ghcr.io/astral-sh/uv:python3.13-trixie-slim
 - ghcr.io/astral-sh/uv:python3.12-trixie-slim
 - ghcr.io/astral-sh/uv:python3.11-trixie-slim
 - ghcr.io/astral-sh/uv:python3.10-trixie-slim
 - ghcr.io/astral-sh/uv:python3.9-trixie-slim

As with the distroless image, each derived image is published with uv version tags as ghcr.io/astral-sh/uv:{major}.{minor}.{patch}-{base} and ghcr.io/astral-sh/uv:{major}.{minor}-{base}, e.g., ghcr.io/astral-sh/uv:0.12.7-alpine.

In addition, starting with 0.8 each derived image also sets UV_TOOL_BIN_DIR to /usr/local/bin to allow `uv tool install` to work as expected with the default user.

For more details, see the GitHub Container page.

Installing uv

Use one of the above images with uv pre-installed or install uv by copying the binary from the official distroless Docker image:

```
FROM python:3.12-slim-trixie
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
```

Or, with the installer:

```
FROM python:3.12-slim-trixie
```

```
# The installer requires curl (and certificates) to download the release archive
```

```
RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates
```

```
# Download the latest installer
ADD https://astral.sh/uv/install.sh /uv-installer.sh

# Run the installer then remove it
RUN sh /uv-installer.sh && rm /uv-installer.sh

# Ensure the installed binary is on the `PATH`
ENV PATH="/root/.local/bin/:$PATH"
```

Note this requires curl to be available.

In either case, it is best practice to pin to a specific uv version, e.g., with:

```
COPY --from=ghcr.io/astral-sh/uv:0.12.7 /uv /uvx /bin/
```

!!! tip

While the Dockerfile example above pins to a specific tag, it's also possible to pin a specific SHA256. Pinning a specific SHA256 is considered best practice in environments that require reproducible builds as tags can be moved across different commit SHAs.

```
```Dockerfile
e.g., using a hash from a previous release
COPY --from=ghcr.io/astral-sh/
uv@sha256:2381d6aa60c326b71fd40023f921a0a3b8f91b14d5db6b90402e65a635053709 /uv /uvx /
bin/
```
```

Or, with the installer:

```
ADD https://astral.sh/uv/0.12.7/install.sh /uv-installer.sh
```

Installing a project

If you're using uv to manage your project, you can copy it into the image and install it:

```
# Copy the project into the image
COPY . /app
```

```
# Disable development dependencies
ENV UV_NO_DEV=1
```

```
# Sync the project into a new environment, asserting the lockfile is up to date
WORKDIR /app
RUN uv sync --locked
```

!!! important

It is best practice to add ``.venv`` to a `[`.dockerignore` file](https://docs.docker.com/build/concepts/context/#dockerignore-files)` in your repository to prevent it from being included in image builds. The project virtual environment is dependent on your local platform and should be created from scratch in the image.

Then, to start your application by default:

```
# Presuming there is a `my_app` command provided by the project
CMD ["uv", "run", "my_app"]
```

!!! tip

It is best practice to use [intermediate layers](#intermediate-layers) separating installation of dependencies and the project itself to improve Docker image build times.

See a complete example in the `uv-docker-example` project.

Using the environment

Once the project is installed, you can either *activate* the project virtual environment by placing its binary directory at the front of the path:

```
ENV PATH="/app/.venv/bin:$PATH"
```

Or, you can use `uv run` for any commands that require the environment:

```
RUN uv run some_script.py
```

!!! tip

Alternatively, the `[`UV_PROJECT_ENVIRONMENT` setting](../../concepts/projects/config.md#project-environment-path)` can be set before syncing to install to the system Python environment and skip environment activation entirely.

Using installed tools

To use installed tools, ensure the tool bin directory is on the path:

```
ENV PATH=/root/.local/bin:$PATH
```

```
RUN uv tool install cowsay
```

```
$ docker run -it $(docker build -q .) /bin/bash -c "cowsay -t hello"
```

```
| _____ |
| hello |
|=====|
| \
| \
|  ^ ^
| (oo)\_____)
| (__)\\       )\/\
|    ||----w |
|    ||     ||
```

!!! note

The tool bin directory's location can be determined by running the ``uv tool dir --bin`` command in the container.

Alternatively, it can be set to a constant location:

```
```dockerfile title="Dockerfile"
ENV UV_TOOL_BIN_DIR=/opt/uv-bin/
```
```

Developing in a container

When developing, it's useful to mount the project directory into a container. With this setup, changes to the project can be immediately reflected in a containerized service without rebuilding the

image. However, it is important *not* to include the project virtual environment (.venv) in the mount, because the virtual environment is platform specific and the one built for the image should be kept.

Mounting the project with docker run

Bind mount the project (in the working directory) to /app while retaining the .venv directory with an anonymous volume:

```
$ docker run --rm --volume ./app --volume /app/.venv [...]
```

!!! tip

The `--rm` flag is included to ensure the container and anonymous volume are cleaned up when the container exits.

See a complete example in the `uv-docker-example` project.

Configuring watch with docker compose

When using Docker compose, more sophisticated tooling is available for container development. The watch option allows for greater granularity than is practical with a bind mount and supports triggering updates to the containerized service when files change.

!!! note

This feature requires Compose 2.22.0 which is bundled with Docker Desktop 4.24.

Configure watch in your Docker compose file to mount the project directory without syncing the project virtual environment and to rebuild the image when the configuration changes:

```
services:
  example:
    build: .

    # ...

  develop:
    # Create a `watch` configuration to update the app
    #
    watch:
      # Sync the working directory with the `/app` directory in the container
      - action: sync
        path: .
        target: /app
      # Exclude the project virtual environment
      ignore:
        - .venv/

    # Rebuild the image on changes to the `pyproject.toml`
    - action: rebuild
      path: ./pyproject.toml
```

Then, run `docker compose watch` to run the container with the development setup.

See a complete example in the `uv-docker-example` project.

Optimizations

Compiling bytecode

Compiling Python source files to bytecode is typically desirable for production images as it tends to improve startup time (at the cost of increased installation time and image size).

To enable bytecode compilation, use the `--compile-bytecode` flag:

```
RUN uv python install --compile-bytecode
RUN uv sync --compile-bytecode
```

Alternatively, you can set the `UV_COMPILE_BYTECODE` environment variable to ensure that all commands within the Dockerfile compile bytecode:

```
ENV UV_COMPILE_BYTECODE=1
```

!!! note

`uv` will only compile the standard library of `_managed_` Python versions during ``uv python install``. The distributor of unmanaged Python versions decides if the standard library is pre-compiled. For example, the official ``python`` image will not have a compiled standard library.

Caching

A cache mount can be used to improve performance across builds:

```
ENV UV_LINK_MODE=copy
```

```
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync
```

Changing the `UV_LINK_MODE` silences warnings about not being able to link files since the cache and sync target are on separate file systems.

If you're not mounting the cache, image size can be reduced by using the `--no-cache` flag or setting `UV_NO_CACHE`.

By default, managed Python installations are not cached before being installed. Setting `UV_PYTHON_CACHE_DIR` can be used in combination with a cache mount:

```
ENV UV_PYTHON_CACHE_DIR=/root/.cache/uv/python
```

```
RUN --mount=type=cache,target=/root/.cache/uv \
    uv python install
```

!!! note

The cache directory's location can be determined by running the ``uv cache dir`` command in the container.

Alternatively, the cache can be set to a constant location:

```
```dockerfile title="Dockerfile"
ENV UV_CACHE_DIR=/opt/uv-cache/
```
```

Intermediate layers

If you're using `uv` to manage your project, you can improve build times by moving your transitive dependency installation into its own layer via the `--no-install` options.

`uv sync --no-install-project` will install the dependencies of the project but not the project itself. Since the project changes frequently, but its dependencies are generally static, this can be a big time saver.

```
# Install uv
FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

# Change the working directory to the `app` directory
WORKDIR /app

# Install dependencies
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv sync --locked --no-install-project

# Copy the project into the image
COPY . /app

# Sync the project
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked
```

Note that the `pyproject.toml` is required to identify the project root and name, but the project *contents* are not copied into the image until the final `uv sync` command.

!!! tip

If you want to remove additional, specific packages from the sync, use `--no-install-package <name>`.

Intermediate layers in workspaces

If you're using a workspace, then a couple changes are needed:

- Use `--frozen` instead of `--locked` during the initial sync.
- Use the `--no-install-workspace` flag which excludes the project *and* any workspace members.

```
# Install uv
FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

WORKDIR /app

RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv sync --frozen --no-install-workspace

COPY . /app

RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked
```

`uv` cannot assert that the `uv.lock` file is up-to-date without each of the workspace member `pyproject.toml` files, so we use `--frozen` instead of `--locked` to skip the check during the initial sync. The next sync, after all the workspace members have been copied, can still use `--locked` and will validate that the lockfile is correct for all workspace members.

Non-editable installs

By default, uv installs projects and workspace members in editable mode, such that changes to the source code are immediately reflected in the environment.

uv sync and uv run both accept a --no-editable flag, which instructs uv to install the project in non-editable mode, removing any dependency on the source code.

In the context of a multi-stage Docker image, --no-editable can be used to include the project in the synced virtual environment from one stage, then copy the virtual environment alone (and not the source code) into the final image.

For example:

```
# Install uv
FROM python:3.12-slim AS builder
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

# Use the system Python across both stages
ENV UV_PYTHON_DOWNLOADS=0

# Change the working directory to the `app` directory
WORKDIR /app

# Install dependencies
RUN --mount=type=cache,target=/root/.cache/uv \
    --mount=type=bind,source=uv.lock,target=uv.lock \
    --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
    uv sync --locked --no-install-project --no-editable

# Copy the project into the intermediate image
COPY . /app

# Sync the project
RUN --mount=type=cache,target=/root/.cache/uv \
    uv sync --locked --no-editable

FROM python:3.12-slim

# Copy the environment, but not the source code
COPY --from=builder /app/.venv /app/.venv

# Run the application
CMD ["/app/.venv/bin/hello"]
```

Using uv temporarily

If uv isn't needed in the final image, the binary can be mounted in each invocation:

```
RUN --mount=from=ghcr.io/astral-sh/uv,source=/uv,target=/bin/uv \
    uv sync
```

Using the pip interface

Installing a package

The system Python environment is safe to use in this context, since a container is already isolated. The --system flag can be used to install in the system environment:

```
RUN uv pip install --system ruff
```

To use the system Python environment by default, set the `UV_SYSTEM_PYTHON` variable:

```
ENV UV_SYSTEM_PYTHON=1
```

Alternatively, a virtual environment can be created and activated:

```
RUN uv venv /opt/venv
# Use the virtual environment automatically
ENV VIRTUAL_ENV=/opt/venv
# Place entry points in the environment at the front of the path
ENV PATH="/opt/venv/bin:$PATH"
```

When using a virtual environment, the `--system` flag should be omitted from `uv` invocations:

```
RUN uv pip install ruff
```

Installing requirements

To install requirements files, copy them into the container:

```
COPY requirements.txt .
RUN uv pip install -r requirements.txt
```

Installing a project

When installing a project alongside requirements, it is best practice to separate copying the requirements from the rest of the source code. This allows the dependencies of the project (which do not change often) to be cached separately from the project itself (which changes very frequently).

```
COPY pyproject.toml .
RUN uv pip install -r pyproject.toml
COPY . .
RUN uv pip install -e .
```

Verifying image provenance

The Docker images are signed during the build process to provide proof of their origin. These attestations can be used to verify that an image was produced from an official channel.

For example, you can verify the attestations with the GitHub CLI tool `gh`:

```
$ gh attestation verify --owner astral-sh oci://ghcr.io/astral-sh/uv:latest
Loaded digest sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
for oci://ghcr.io/astral-sh/uv:latest
Loaded 1 attestation from GitHub API
```

The following policy criteria will be enforced:

- OIDC Issuer must match:..... <https://token.actions.githubusercontent.com>
- Source Repository Owner URI must match:... <https://github.com/astral-sh>
- Predicate type must match:..... <https://slsa.dev/provenance/v1>
- Subject Alternative Name must match regex: (?i)^<https://github.com/astral-sh/>

✓ Verification succeeded!

sha256:xx was attested by:

| REPO | PREDICATE_TYPE | WORKFLOW |
|--------------|---|--|
| astral-sh/uv | https://slsa.dev/provenance/v1 | .github/workflows/build-docker.yml@refs/heads/main |

This tells you that the specific Docker image was built by the official uv GitHub release workflow and hasn't been tampered with since.

GitHub attestations build on the sigstore.dev infrastructure. As such you can also use the cosign command to verify the attestation blob against the (multi-platform) manifest for uv:

```
$ REPO=astral-sh/uv
$ gh attestation download --repo $REPO oci://ghcr.io/${REPO}:latest
Wrote attestations to file
sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.jsonl.
Any previous content has been overwritten
```

```
The trusted metadata is now available at
sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.jsonl
$ docker buildx imagetools inspect ghcr.io/${REPO}:latest --format
"{{json .Manifest}}" > manifest.json
$ cosign verify-blob-attestation \
  --new-bundle-format \
  --bundle "$(jq -r .digest manifest.json).jsonl" \
  --certificate-oidc-issuer="https://token.actions.githubusercontent.com" \
  --certificate-identity-regexp="^https://github\.com/${REPO}/.*" \
  <(jq -j '.|del(.digest,.size)' manifest.json)
Verified OK
```

!!! tip

These examples use `latest`, but best practice is to verify the attestation for a specific version tag, e.g., `ghcr.io/astral-sh/uv:0.12.7`, or (even better) the specific image digest, such as `ghcr.io/astral-sh/uv:0.5.27@sha256:5adef09a5a526f380237408032a9308000d14d5947eafa687ad6c6a2476787b4f`.